

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 05 · LECTURE NOTES

Sketches and Streams

Answers from data you cannot store.

MODULE OUTCOMES

- Explain the streaming model and its constraints.
- Implement and analyse a count-min sketch.
- Explain HyperLogLog and distinct counting.
- Explain reservoir sampling and its correctness.
- Choose a sketch for a problem and state its error bound.

1 The streaming model

THE STREAMING MODEL, AND WHAT IT FORCES

Items arrive one at a time, you may not store them, and the available memory is polylogarithmic in the stream length. One pass, or sometimes a small constant number.

Which forces approximation. Exact distinct counting provably requires space linear in the number of distinct items (by a communication-complexity argument), so every sublinear-space answer is necessarily approximate — **and the model's value is that the approximation comes with a bound** rather than being a hope.

And the guarantees take the (ϵ, δ) form: the answer is within a relative error ϵ with probability at least $1 - \delta$, with the memory requirement depending on both parameters.

So the design question is always the same: how much memory, for what accuracy, at what confidence — and all three are tunable, which is precisely what makes sketches engineering tools rather than theoretical curiosities. **You pick the point on the surface that your application needs.**

Sketch	What it answers	Memory
Count-min	Item frequencies, and heavy hitters.	$O((1/\epsilon) \log(1/\delta))$ counters (§2).
HyperLogLog	The number of distinct items.	About 1.5 KB for 2% error, at any cardinality — see the note.
Bloom filter	Membership, with one-sided error.	About 10 bits per item for 1% false positives.
AMS / tug-of-war	The second frequency moment, and self-join size.	$O((1/\epsilon^2) \log(1/\delta))$ — note the squared dependence, which is worse.

Sketch	What it answers	Memory
Reservoir sample	A uniform random sample of size k .	$O(k)$ — and it is <i>exact</i> , not approximate (§3).
t-digest, KLL	Quantiles and percentiles.	$O((1/\epsilon) \log)$ — and KLL is the one with proved bounds, while t-digest is the one in wide use.

HyperLogLog's memory being independent of the cardinality is the result that makes distinct counting at scale possible at all — 1.5 kilobytes whether the stream contains a thousand distinct items or a billion, which is why it is deployed essentially universally for unique-visitor and cardinality metrics. The scale-independence is what makes people adopt it, more than the error bound.

2 Count-min

A $d \times w$ table of counters, and d independent hash functions mapping items into $[w]$.

```
UPDATE(x, c):  for each row  $i$ , add  $c$  to  $\text{table}[i][h_i(x)]$ 
QUERY(x):     return the MINIMUM over rows  $i$  of
                $\text{table}[i][h_i(x)]$ 
```

WHY THE MINIMUM: each cell holds x 's true count PLUS the counts of every other item that happens to hash to that cell. So EVERY row OVERESTIMATES, and the minimum across rows is the least bad overestimate available.

-> ONE-SIDED ERROR. The answer is never too small, which is a useful property when thresholding.

THE BOUND: with $w = e/\epsilon$ and $d = \ln(1/\delta)$,
 $\text{estimate} \leq \text{true count} + \epsilon * (\text{total stream weight})$
 with probability at least $1 - \delta$.

NOTE CAREFULLY WHAT THE ERROR IS RELATIVE TO: the TOTAL stream weight, not x 's own count. So count-min is excellent for HEAVY HITERS and essentially useless for rare items -- an item with true count 3 in a stream of total weight 10^9 will have an estimate completely dominated by noise.

WHICH IS THE QUESTION TO ASK OF ANY SKETCH: the error is relative to WHAT?

3 Distinct counting, and sampling

HYPERLOGLOG COUNTS DISTINCT ITEMS BY LOOKING AT LEADING ZEROS

Hash each incoming item and record the maximum number of leading zeros observed in any hash value. If you have seen a hash with k leading zeros, you have probably processed about 2^k distinct items — because a random hash has k leading zeros with probability 2^{-k} .

And duplicates do not affect the estimate at all, because hashing is deterministic and a repeated item produces the same hash and therefore the same leading-zero count — **which is exactly what makes it a distinct counter rather than a counter**. The idea is genuinely elegant.

A single such estimate is extremely noisy, so use many registers: use the first few hash bits to select one of m registers, record the leading-zero count in each, and **combine with a harmonic mean**, which suppresses the influence of individual large outliers.

Giving about 2% relative error from roughly 1.5 kilobytes, at any cardinality — and the memory is independent of the count because each register stores an *exponent* (a small integer, 6 bits) rather than a count. That substitution of an exponent for a number is the whole trick.

RESERVOIR SAMPLING, WHICH IS EXACT

To maintain a uniform random sample of k items from a stream whose length you do not know in advance: keep the first k items, then for the n -th item ($n > k$), replace a uniformly chosen reservoir slot with probability k/n .

The result is exactly uniform — a short induction shows that after n items every item seen has probability exactly k/n of being in the reservoir, which is what uniformity means.

So this is not a sketch in the approximate sense: the sample is exactly correct, and only the *choice* of which items are in it is random. No ϵ and no δ appear.

Which makes it the most broadly useful tool in the module — a uniform sample lets you estimate essentially anything (with error governed by Module 02's bounds on the sample), **rather than answering one specific pre-chosen question like the other sketches do**. So it is worth trying first (§4), and a specialised sketch is worth it when the sample is too small to resolve what you need.

4 Choosing and using a sketch

- Ask what the error is relative to. Count-min's error is relative to the *total stream weight*; HyperLogLog's is relative to the *cardinality*; a sample's is relative to the quantity being estimated — **completely different guarantees**, and this is the question that decides whether a sketch answers your question at all.
- Check mergeability. Count-min, HyperLogLog, and Bloom filters all merge (cellwise maximum or sum, register-wise maximum, bitwise OR) — **which is what makes them usable in a distributed aggregation** (CSCE 678 Module 04), where each shard sketches locally and the sketches are combined.

- **Check the error direction. Count-min overestimates and never underestimates**, which matters a great deal when you threshold on the result — you will get false heavy hitters and never miss a real one.
- **Measure the achieved error against the bound. It is usually far better**, because the bound is worst-case over all streams — **and reporting the measured error alongside the guarantee is more useful than reporting the guarantee alone** (Module 13 §1).
- **And consider a plain sample first. Reservoir sampling plus exact computation on the sample answers a great many questions** that a specialised sketch would also answer, with far less machinery and a guarantee you can derive in two lines. **Mergeability is the property that makes sketches infrastructure**, and it is the one to check first when designing for a distributed setting.

WHY SKETCHES ARE THE MOST USED MATERIAL IN THIS COURSE

Every large-scale monitoring, analytics, or observability system is built from these — cardinality estimation for unique counts, heavy hitters for top-k, quantile sketches for latency percentiles, Bloom filters for negative lookups — **at volumes where exact answers are simply not available**.

And the guarantees are the selling point: '2% error with 99% confidence from 1.5 kilobytes' is a specification an engineer can design against, which an unquantified approximation is not. **The bound is the product**.

So this module is where Module 02's concentration bounds pay off most visibly — every error bound in §1's table is a Chernoff or Chebyshev argument, and deriving one yourself is the fastest way to make the toolkit feel useful rather than abstract.

And the honest caveat: the bounds are worst-case over all streams, and your stream is not adversarial — so measure, and expect to do considerably better than the guarantee, which is CSCE 637 Module 01 §3's worst-case qualification arriving in a setting where it is good news.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cormode & Yi — Small Summaries for Big Data (free)	Free in full, and the reference for this whole module — every sketch in §1's table, with its bound and its merge operation.
Cormode & Muthukrishnan — An Improved Data Stream Summary: The Count-Min Sketch (free)	§2's sketch, in the original , with the bound derived.
Flajolet et al. — HyperLogLog (free)	§3's method , including why the harmonic mean is the right combiner.
Vitter — Random Sampling with a Reservoir (free)	§3's reservoir sampling , with the uniformity proof and the faster variants.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a count-min sketch** and measure its error on a real stream against the derived bound.
- 2 **Confirm the error is one-sided** by comparing estimates against exact counts.
- 3 **Query a rare item** and report how bad the estimate is. Relate it to the total weight.
- 4 **Implement HyperLogLog** with 1024 registers and measure the error at several cardinalities.
- 5 **Confirm the memory is independent of the cardinality.**
- 6 **Implement reservoir sampling** and verify the uniformity empirically over many runs.
- 7 **Use the sample to estimate three different quantities** and bound each error with Module 02's tools.
- 8 **Merge two count-min sketches** and two HLL sketches, and verify the results match sketching the combined stream.
- 9 **Compare a sketch against a sample** for one question, and report which was better.
- 10 **Measure the achieved error against the bound** for every sketch you implemented, and report the ratio.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the streaming model's constraints and what they force.
- 2 What is the (ϵ, δ) form, and what is tunable?
- 3 Name six sketches and what each answers.
- 4 Why is HyperLogLog's memory independent of the cardinality?
- 5 Describe count-min and explain why the minimum is taken.
- 6 What is count-min's error relative to, and what follows?
- 7 Explain HyperLogLog's leading-zero idea and why duplicates do not matter.
- 8 Why is reservoir sampling exact, and why is it the most broadly useful?
- 9 Give five practical checks when choosing a sketch.
- 10 Why is mergeability the key infrastructure property?